ULB ECOLE POLYTECHNIQUE DE BRUXELLES

Group Project: Part 2
2024

INFO-H-419: Data Warehouses

TPC-DI: Data Integration Benchmark

Alfio Cardillo, Kristóf Balázs, Stefanos Kypritidis and Josu Bernal

Contents

1 Introduction **2**
1.1 What is PostgreSQL? . . . 2
1.2 What is a benchmark for a RDBMS? . . . 2
1.3 Aim and objectives . . . 2
1.4 GitHub repository . . . 2

2 Benchmark Overview **3**
2.1 Source Data Models and Load Types . . . 3
2.2 Destination Data Model . . . 5
2.3 Transformations . . . 5

3 Technical specifications **6**
3.1 Software specifications . . . 6
3.1.1 Apache Airflow . . . 6
3.2 Hardware specifications . . . 7
3.3 Postgres configuration . . . 7

4 Improvements and optimization **9**
4.1 Reproducibility enhancement . . . 9
4.2 Processing area . . . 9
4.3 History tracking . . . 12
4.3.1 Approaches with DimAccount . . . 14
4.4 Graph dependencies . . . 17
4.5 Incremental updates . . . 19
4.6 Optimization . . . 19
4.7 Other small fixes . . . 22
4.7.1 LTRIM . . . 22
4.7.2 Coalesce . . . 23
4.8 Audit Phase . . . 23
4.8.1 FactWatches SK CustomerID & SK SecurityID . . . 26
4.8.2 DimCustomer TaxID . . . 26
4.8.3 DimSecurity IsCurrent . . . 26
4.8.4 Batches Checks . . . 27
4.9 Batch Validation Query . . . 27

5 Results and discussion **28**
5.1 Performance results . . . 28

6 Conclusion **31**

1 Introduction

This project focuses on evaluating the performance of PostgreSQL as a data warehouse through the TPC-DI benchmark.

1.1 What is PostgreSQL?

PostgreSQL is an open-source relational database management system (RDBMS) that emphasizes extensibility and SQL compliance. It was developed at the University of California, Berkeley and is known for its robustness, reliability, and performance.

1.2 What is a benchmark for a RDBMS?

A benchmark for a Relational Database Management System (RDBMS) is a standardized test used to evaluate and compare the performance and scalability of different database systems. These benchmarks provide a scalable dataset and a set of queries to assess how well a RDBMS performs under specific workloads.

1.3 Aim and objectives

The TPC-DI Benchmark is a performance test for evaluating tools that move and integrate data across systems, commonly known as Data Integration (DI) or Extract, Transform, Load (ETL) tools. The benchmark involves manipulating a specific volume of data to prepare it for a Data Warehouse. It includes data extracted from an Online Transaction Processing system, transformed alongside data from additional sources, and then loaded into the Data Warehouse.

1.4 GitHub repository

As the reader will see throughout this report, we reference our GitHub repository multiple times to avoid unnecessary explanations that do not contribute value to our findings. For those interested in exploring the insights of our work or replicating it, we have included the repository link here: `https://github.com/Carda01/tpcdi-benchmark-postgres`

2 Benchmark Overview

In this section, we will summarize the benchmark pipeline, which is explained in detail in the official documentation.

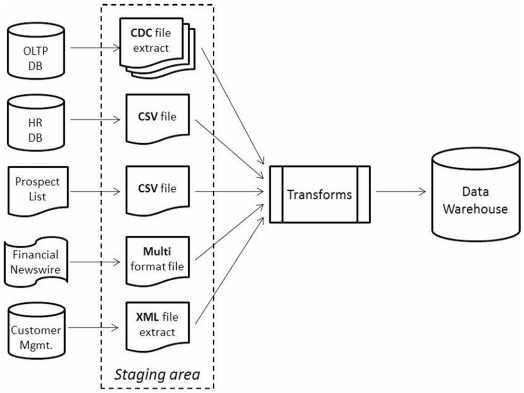


Figure 1: Conceptual model of the benchmark

The data model for the TPC-DI benchmark represents a retail brokerage. Online Transaction Processing (OLTP) data is combined with information from other sources to create the data warehouse. The conceptual model for this benchmark is illustrated in Figure 1. It is important to note that this benchmark assumes the data is already in the staging area, and as a result, the extraction steps are not evaluated (the data is generated beforehand). Additionally, the creation and preparation of the data warehouse will not be timed.

2.1 Source Data Models and Load Types

In this benchmark two different types of data loading are being evaluated, Historic Load and Incremental Update, and each has its own source data to load into the data warehouse:

1. *Historic Load:* A data warehouse is created from historical records. Initially, the destination tables are empty and are populated with new data, while the source files may have varying ordering properties. The Historical Load phase can run for as long as necessary to completely process the data, and its duration is used to compute the benchmark metric.

2. *Incremental Update:* This process represents the loading of new data into an existing data warehouse. The benchmark simulates a daily update. The input files from the OLTP database are modeled as Change Data Capture (CDC) extracts, which display the changes in table data since the last extract. The incremental

update occurs in two phases to ensure repeatability, and each phase is required to be completed within 30 to 60 minutes. Both phases are timed for use in the computation of the benchmark metric.

Now that we have already explained both types of loads, let's study the data sources.

Files for the Historical Load are full extracts with all table rows. On the other hand, Incremental Updates use CDC extracts, which include "CDC_FLAG" and "CDC_DSN" columns. The CDC_FLAG indicates if a row was inserted (I), updated (U), or deleted (D) since the last update, but does not specify what changed in updated rows. Rows that haven't changed will not appear in the CDC extract. The CDC_DSN is a monotonically increasing sequence number unique to the source database, indicating the order in which changes occurred, with rows sorted by this value.

The following files represent the data coming from the OLTP systems and reference data:

- Account.txt (Incremental Update)
- Customer.txt (Incremental Update)
- Trade.txt (Historical Load and Incremental Update)
- TradeHistory.txt (Historical Load)
- CashTransaction.txt (Historical Load and Incremental Update)
- HoldingHistory.txt (Historical Load and Incremental Update)
- DailyMarket.txt (Historical Load and Incremental Update)
- WatchItem.txt (Historical Load and Incremental Update)
- HR.csv (Historical Load)
- Prospects.csv
- FINWARE files (Historical Load)
- CustomerMgmt.xml (Historical Load)
- Date.txt (Historical Load)
- Time.txt (Historical Load)
- Industry.txt (Historical Load)
- StatusType.txt (Historical Load)
- TaxRate.txt (Historical Load)
- TradeType.txt (Historical Load)

2.2 Destination Data Model

The following picture explains visually the Data Warehouse schema where the data will be stored.

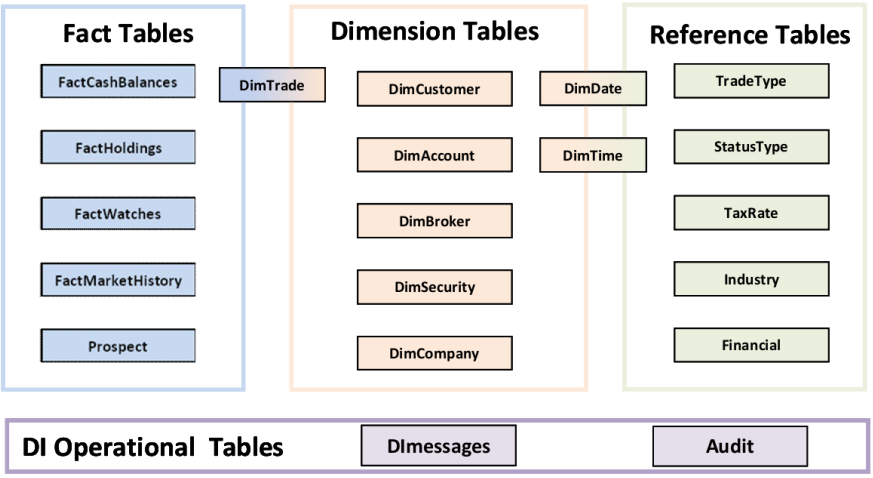


Figure 2: Pictorial overview of the Data Warehouse Tables

2.3 Transformations

The data operations that must be done to prepare and load data into the Data Warehouse are:

- Conversion of data from character representations to data types compatible with the Data Warehouse specification.
- Lookups of business keys to obtain surrogate keys for the Data Warehouse.
- Merging or formatting multiple fields into one, or splitting one field into multiple.
- Checking data for errors or for adherence to business rules.
- Detecting changes in dimension data, and applying appropriate tracking mechanisms (retaining history or overwriting).
- Detecting changes in fact data, and journaling updates to reflect current stat.

3 Technical specifications

In this section, we will outline the software and hardware used to conduct this benchmark, including the versions of the tools and the technical specifications of the machine on which the evaluation was performed. This information provides essential context for our benchmark, as the results are highly dependent on the machine executing the tests. By including these details, we aim to help readers better understand the significance of the results presented.

3.1 Software specifications

In this subsection, we display the specifications of the tools used for this project.

Tool	Version	Description
PostgreSQL	16.4	This is the tool being evaluated. It is a relational database management system (RDBMS) known for its extensibility.
Docker Desktop	4.37.0	We utilized this tool to ensure reproducibility across all operating systems. It functions as a container-based application launcher.
Python	3.10.11	We used it to load the database and execute queries. It is a popular programming language.
Jupyter Notebook	7.2.0	IDE for Python and iPython notebook.
MS PowerBI	latest	Power BI was utilized to create visualizations of benchmarking results for further analysis.
Git-GitHub	latest	Git and GitHub Desktop were used to share code files between the team members and to post the final repository.
Apache-Airflow	2.3.0	The ETL process in this project was implemented as a workflow using Apache Airflow.

3.1.1 Apache Airflow

The ETL process implemented in this project was defined using Apache Airflow. To enhance understanding and prevent future confusion, we find it important to review the main concepts of this software.

Apache Airflow is a platform for programming, scheduling, and monitoring workflows. It allows users to define workflows as code in Python, which can then be executed on a specified schedule or triggered manually. Airflow is highly extensible, enabling the use of custom operators, hooks, and plugins. In this project, we utilized this software to define the ETL process.

A Directed Acyclic Graph (DAG) is a collection of tasks arranged to show their relationships and dependencies. It serves as a representation of a workflow. For instance, a data pipeline that includes steps such as data extraction, transformation, and loading (ETL) can be depicted as a DAG.

- Directed: Each task in a DAG points to the next in the sequence.
- Acyclic: It cannot have cycles; no task can be a parent of itself.

A task represents a single unit of work within a DAG. Each task performs a specific operation, such as running a Python script, executing a SQL query, or transferring data between systems.

The DAG defines the workflow's overall structure. Tasks define the specific actions within the workflow.The tasks in the DAG are connected by dependencies to dictate the order of execution.

3.2 Hardware specifications

In this subsection, we display the hardware specifications of the machine used for this project.

Processor	Intel® Core™ i5-13500H, 12C (4P + 8E) / 16T, P-core 2.6, 4.7GHz, E-core 1.9 / 3.5GHz, 18MB
RAM	32.0 GB LPDDR5-5200
GPU	NVIDIA GeForce RTX 3050 6GB GDDR6 Laptop GPU
Storage	1TB SSD M.2 2242 PCIe 4.0x4 NVMe

3.3 Postgres configuration

In this subsection, we will go into technical details of the PostgreSQL configuration that we have used. We modified this by changing the "postgresql.conf" file, with the purpose of improving the allocation of resources of PostgreSQL to increase performance and take advantage of our hardware specifications. This information could be useful for anyone wanting to reproduce our results. We will only list the changes made because they are pretty self-explanatory.

- Memory
  - shared_buffers = 4GB (original is 128MB)
  - work_mem = 16MB (original is 4MB and is commented)

  - maintenance_work_mem = 1GB (original is 64MB and is commented)

- Asynchronous Behavior
  - max_worker_processes = 16 (original is 8 and is commented)
  - max_parallel_workers_per_gather = 8 (original is 2 and is commented)
  - max_parallel_maintenance_workers = 8 (original is 2 and is commented)
  - max_parallel_workers = 16 (original is 8 and is commented)
- Planner Cost Constants (This will play a big role later)
  - random_page_cost = 1.1 (original is 4.0 and is commented, uncomment also seq_page_cost)
  - effective_cache_size = 24GB (original 4GB and commented)

4 Improvements and optimization

In this section, we will explain the work we did in the ETL process performed in this project. First, we want to clarify that we started this project based on the work done by previous students on historical loading [1] and incremental updates [2]. We believe it is unhelpful to repeat work that has already been accomplished, and our goal each year should be to identify and implement improvements.

We spent much time trying to understand where we could improve their solution and how their process worked.

By reading the specification and understand thoroughly both what the specification focused on and their project, we came up with some changes that will be described below.

Some of them will have a great impact on the Result chapter below.

4.1 Reproducibility enhancement

In this subsection, we will explain how we enhanced the reproducibility of this project so that any future person who wants to run this benchmark on PostgreSQL will find it straightforward on any operative system. This section will detail how we improved the reproducibility and setup for future generations.

The process of generating data for TCP-DI is straightforward, so we did not focus on that aspect. Instead, we have prepared a detailed README.md file that explains all the necessary setup steps. Additionally, we have created two Python notebooks that automate the entire setup process, making it as simple as clicking through each cell.

The first notebook, called **setup.ipynb**, is responsible for downloading, installing, and configuring Airflow. It executes several terminal commands based on the operating system, to configure the required Docker containers for running Airflow. The second notebook called, **create_sf.ipynb**, handles the creation of DAGs based on specified scale factors. We have developed various DAG templates, and this notebook copies them while adjusting the necessary fields to ensure they function correctly. This adjustment is essential because the DAGs and processes need to access different folders on the computer depending on the scale factor and the operating system, and these paths change with each scale factor.

To sum up, all these files can be found in our GitHub repository, so we don't think it's necessary to explain the code and process in more detail.

4.2 Processing area

One of the issue we found was the concept of **Staging Area**.

In the specifications[4] the staging area is described as the single directory that holds the **Source Data** and the format of these source data is the original one, as it can be inferred from figure 3. Another important aspect of the **Staging area** is that, copying data from the Source folder to the Staging area, shall not be timed as part of the benchmark, which should instead calculate the performance of the loading from the staging area to the Data Warehouse.

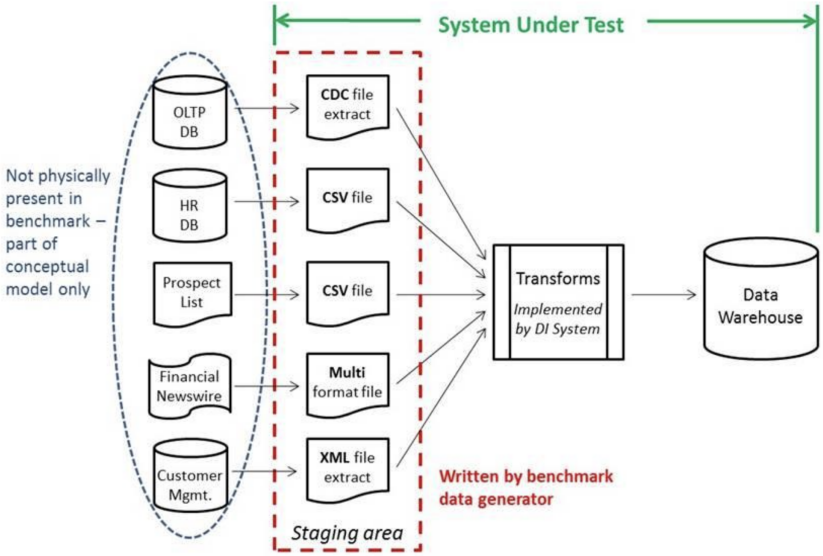


Figure 3: Definition of staging area

So basically, after generating the data, we defined our **Staging area** to be the directory inside the airflow dags folder, named data.

For all of the reasons above we've decided that naming the first auxiliary schema the old project used where they loaded the data from their different formats to relational tables, should not be called **staging**, as in this "staging" area, our data is already in relational form and the loading from source to "staging" is part of the benchmark, as it is included in the DAG.

In this section we will refer with "staging" to that area they had defined, in contrast with the **Staging area** which is instead what we've defined above, following the tpcdi specs.

Our solution was to rename this "staging" area to what we called **processing** area, as it is the area where our data is already loaded as relational data, and where we can do calculations or creation of data structures, before actually being inserted in our final tables, those that appear in the **master** schema.

After that we were able to solve some issues that was generated from the confusion in the concept of staging area.

First of all we've noticed that the following tables were loaded first in their "staging" area and then directly copied with an insert from the "staging" to the **master**.

1. **master.tradetype**
2. **master.statustype**
3. **master.taxrate**
4. **master.industry**
5. **master.dimdate**
6. **master.dimtime**

Actually while for the first 4 this was done with an insert into ... select * from, the last two, casted the datevalue and the timevalue to date and time respectively while inserting.

We've noticed that this is not useful, as PostgreSQL will automatically do this casting from text to date/time if the second table has for type of that column the correct type.

In general, we noticed that inserting first into "staging" and then copying to **master** is useless, and was probably due to a wrong understanding in the concept of **Staging area**.

Hence we've removed the definition of these tables in our processing area, and we've substituted the old tasks

- **load_master_tradetype**
- **load_master_statustype**
- **load_master_taxrate**
- **load_master_industry**
- **transform_load_master_dimdate**
- **transform_load_master_dimtime**

with a new **load_directly_master_tables**, which indeed loads from the **Staging area** to the **master** schema those tables which do not require any transformation and are ready to be uploaded with a **copy** statement.

With this operation we removed the duplication of copying and actually we also removed the part that may take the most, i.e. the insert into, as we now load the data by copying.

We say so because by reading this article [3] we discoverded that the copy operation is the fastest in PostgreSQL, and both outperforms row by row insertion and bulk insertion.

4.3 History tracking

When going through the specs[4], we've noticed that the old project did not implement neither what is specified in 4 nor the part of 5

4.4.1.2 When a record with a natural key that does not exist in the dimension table is processed, the following transformations are performed:

1. A unique surrogate key value must be assigned and included in the inserted record.
2. The current indicator field, IsCurrent, is set to TRUE to indicate that this is the current record corresponding to the natural key.
3. The EffectiveDate field is set to a value specified by the transformation, or **Batch Date** if no value is specified.
4. The EndDate field is set to December 31, 9999.

Rationale: The EndDate of the 'current' record is not yet determined. The actual EndDate is set when a new record is inserted and this record is expired. When querying a dimension to find the valid record for a given time, a condition like EffectiveDate <= my_time < EndDate could be used. Using a NULL value for EndDate complicates these sorts of queries as these conditions will be UNKNOWN on current records, so additional logic would need to be added to account for that. To avoid this complication, a date far off into the future is used as the EndDate for current records, which allows a basic date range search to work for all records.

4.4.1.3 When a record with a natural key already present in the dimension table is processed, the following transformations are performed:

1. Update the existing dimension table record for that natural key where IsCurrent is set to TRUE (these updates are known as 'Expiring' the record):
   - The current indicator field, IsCurrent, is set to FALSE to indicate that this is no longer the current record corresponding to the natural key.
   - The EndDate field is set to the EffectiveDate of the new record.

2. After expiring the existing record in the dimension table, a new record is inserted into the dimension table, as described in Clause 4.4.1.2.

Figure 4: Specification for history-tracking dimension tables

4.4.1.6.1 When a history-tracking dimension table contains a surrogate key reference to another dimension table and an update occurs to the referenced dimension table, the referencing table must be updated as well. Specifically, this situation exists for:

- DimAccount, which contains a surrogate key for DimCustomer
- DimSecurity, which contains a surrogate key for DimCompany

Figure 5: Specification for history-tracking dimension tables with references

The tables that needs to support history-tracking are:

- **DimAccount**
- **DimBroker**
- **DimCompany**
- **DimCustomer**
- **DimSecurity**

The issue that we came across, was that 4 and 5 suggest, that the data is inserted row by row and that while inserting we check on the final table if some of the conditions described apply.

But the structure we were working with, at least for the historical load, inserted all rows at the same time.

We decided to keep this structure, as it is without any doubt more efficient than inserting row by row and we achieved historical tracking using a cte with a window function like that of figure 6.

The examples shown below are taken from the tables DimSecurity and DimCompany, but a similar work was done for the other dimensions, except the broker, that at least in the historical load was just copied from the HR.csv and did not require any history-tracking logic.



Figure 6: CTE definition

As we can see we are partitioning our windows with the company key and ordering using the timestamp.

With lead we then take the following timestamp in the sequence. So we basically enhanced the base table, in this case processing.finwire_cmp by adding also the following timestamp for each company id.

An important feature of this method is that in case there is not another company with that same id and with a higher timestamp, it means that we reached the actual company which is now active. In this case the window function will return for such records null in the column next_pts.

We can see how we used this information, when actually creating the new final columns in figure 7

```
case
    when f.next_pts is null
        then true
    else false
end as iscurrent,
1 as batchid,
left(f.pts, 8)::date as effectivedate,
case
        when f.next_pts is null
        then '9999-12-31'::date
        else left(f.next_pts, 8)::date
end as enddate
```

Figure 7: History column definition

Here we are indeed defining iscurrent as true, if the record has no next timestamp, false, if instead, there is a following record which has the same company id.

We do something similar with the enddate, indeed, if the next timestamp is null then we return the max date, defined as "9999-12-31", if instead there is a next timestamp, we convert it to a date by taking the first 8 characters of the string and converting them into a date.

This was done because the input timestamp is like:

$$19670101 - 154842$$

The first 8 characters represent the date, the remaining part is the time.

For the other dimensions we did something similar, we changed of course the input table and the columns we partitioned by.

The issue was now making sure that also 5 was satisfied, and unfortunately, we couldn't achieve this goal, but we will briefly talk of all the approaches we tried by looking at DimAccount and its relation with the DimCustomer, to conclude, that probably a total rework of the logic is needed.

4.3.1 Approaches with DimAccount

First of all we've solved an issue in the **8_transform_load_master_dimaccount.sql** that wrongly differentiate if an account was new or not. Indeed, an account is new when actiontype is 'NEW' but also when we are adding a new account with 'ADDACCT'.

This was important because when we have an update on the customer ('UPDCUST'), the information, of the account are not passed, but a new row in the account should be created with the new customer surrogate key and both the new effective and end dates.

So before inserting into account, the information of the old account when such actiontypes happen should be retrieved, and this is done by joining and coalescing the columns with those rows where the accounts were created i.e. those where action type is 'ADDACCT' or 'NEW', but before only 'NEW' rows were considered.

Here we see a first problem of this structure: in case the account was updated in the meantime, we now produce a row, which has for account information the one of its creation, and that has lost what has happened in the meantime. And it seems quite difficult to keep track of all these changes without a row by row insertion we will propose at the end.

Now going addressing the first issue, we explored how, we approached the problem of an update to the customer.

When action type is 'UPDCUST' but also when it's 'INACT' (disactivate the customer and all the accounts related to it), we noticed that the key of the account ca_id was set to null.

So our idea was to remove that row, and replace it with multiple rows, all having the same data of the row deleted, but with ca_id set to those account that were already, created or added to the customer, at the moment of update. The way we did so is with the query defined in 8.

By doing so, we thought that now the partitioning job we've described in the cte above6 will also cover the cases described in the specification 5.

As now there will be a new next timestamp set for each row that had a following record with an update or with an inactivation of the customer.

We refer to this operation as an expansion, because we are basically expanding the customermgmt table with copies of the same row, indeed as we can see from 8 we are taking all the rows of customermgmt (we noted that with ... in the select) and we are just substituting the ca_id.

```
create table processing.mod_customermgmt as table processing.customermgmt;
DELETE FROM processing.mod_customermgmt
WHERE actiontype in ('UPDCUST', 'INACT') AND ca_id IS NULL;
WITH update_rows AS (
    SELECT
        *
    FROM
        processing.customermgmt
    WHERE
        actiontype = 'UPDCUST'
        or actiontype = 'INACT'
),
valid_ca_ids AS (
    SELECT DISTINCT
        ca_id
    FROM
        processing.customermgmt
    WHERE
        ca_id IS NOT NULL
)

insert into processing.mod_customermgmt
SELECT
    ....
    v.ca_id,
    ...
FROM
    update_rows mt
CROSS JOIN valid_ca_ids v
WHERE
    v.ca_id IN (
        SELECT
            ca_id
        FROM
            processing.customermgmt
        WHERE
            actionts < mt.actionts
            and c_id = mt.c_id
    );
```

Figure 8: Create a new modified customermgmt with the expansion on the update

Unfortunately, this approach also failed, because the rest of the architecture was not able to work with this expansion of the customermgmt processing table, and at that moment we had found other issues, like the one seen above for the new vs old account which discouraged us from continuing pursuing this path.

To conclude, due to lack of time we couldn't explore other possibilities, because the only thing left we could have tried was to start from scratch, but we were in a phase of the project were this was not possible.

Indeed a solution that might work, is to simply follow the suggestion given by the specification, i.e. doing a row by row insertion, or even a batch one when the dependencies allow it, that inserts the rows, in the order of the timestamp they come

with.

This code might also be reusable for the incremental loads later. In this way one can check at any insertion what is the data already stored and change it appropriately.

The issue with this approach is that performance-wise, we might get lower performances also from what we've seen in the article[3].

4.4 Graph dependencies

An issue we found which was quite time-consuming was endless running tasks. Probably due to our optimizations, some tasks ended earlier, this caused, the beginning of other tasks, and apparently when some tasks were executed in parallel, one of them never stopped running.

An example is given in figure 9 where the task **factmarkethistory** for scale factor 10, after running for 2 hours and a half, made us force it to stop.

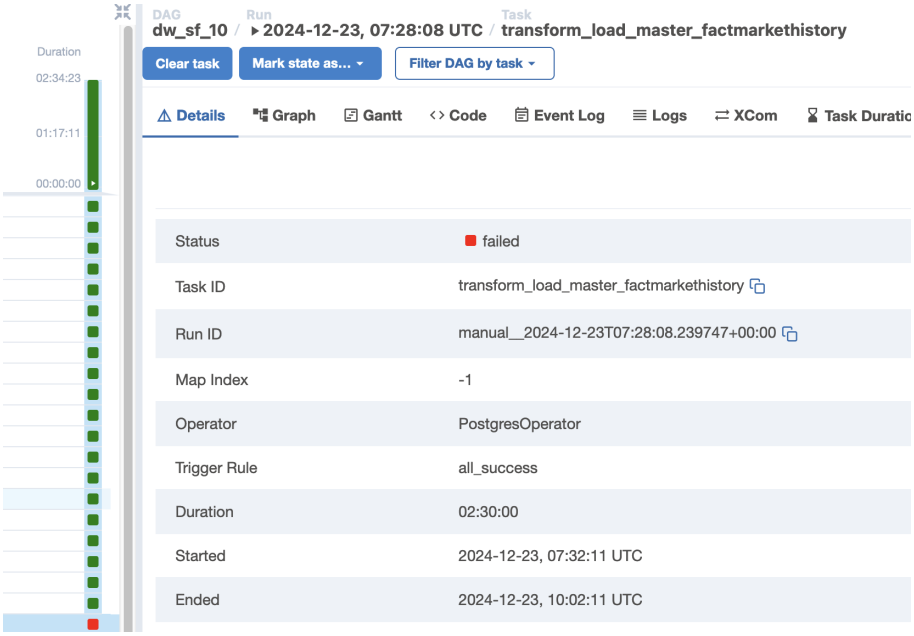


Figure 9: Endless running task example

In those cases we tried unsuccessfully different techniques like:

- Investigate if there was some locking happening on PostgreSQL side, but what we saw, was that just that query was in execution, and apparently no other query was holding some lock.
- Adding sleep time between tasks, but this might have slowed down the process drastically.

- Allocating more resources to PostgreSQL, in case the issue was due to the heavy load, but again, this did not produce any results.

At the end the solution we found, was introducing some other dependencies (graph's edges), forcing the tasks that when run in parallel caused issues, to now run sequentially. This hopefully fixed our issue, with the counter effect of slightly reducing parallelization.

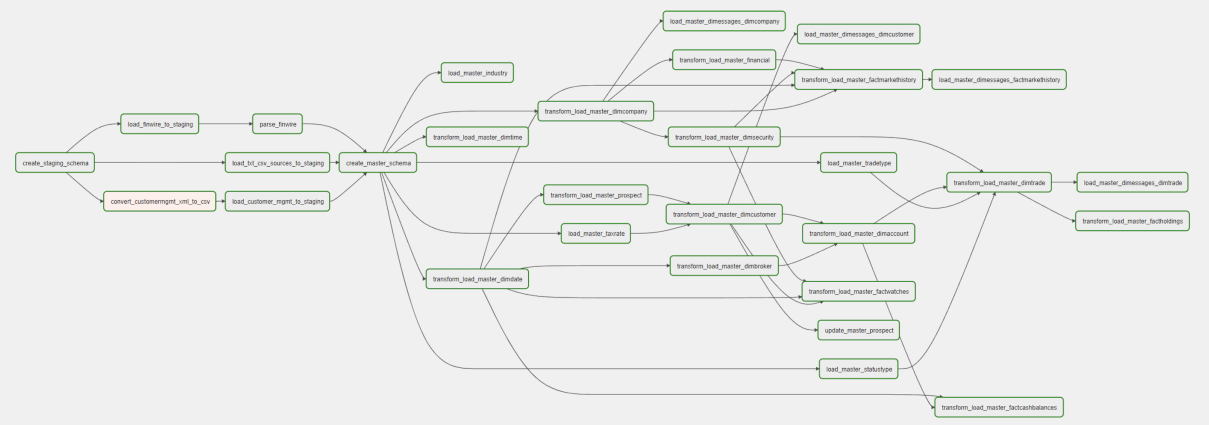


Figure 10: Old DAG

You can see in figure 10 the old graph, while in figure 11, the new graph, with both the new dependencies, renamings and the changes described in section 4.2.



Figure 11: New DAG

We do not know if these issues were present, also in the old project, or if as we hypothesized before, were introduced because of the optimization of some early tasks, that in cascade made the heavier last tasks to run in parallel.

Nor we had clear if they were caused, from Airflow, PostgreSQL, or from the queries, although we checked that no queries where a table was inserted into and selected from, run in parallel, and that all such queries were correctly chained with those inserting into, running before those selecting from the tables.

4.5 Incremental updates

The TPC-DI benchmark requires two Incremental Update phases to ensure repeatability and consistency in the testing process. Building on the foundation established in the previous year [2], our focus was primarily on debugging and refining errors that were identified during the execution of the benchmark. We successfully implemented both of the required incremental updates, which are a mandatory part of the TPC-DI report. During the review of these updates, we uncovered several issues that were not immediately apparent from a straightforward code examination. These problems, which affected the integrity and performance of the updates, will be thoroughly discussed and addressed in the audit phase to ensure the robustness of the final implementation.

It was also observed that in the previous year's implementation, much of the work in the incremental updates was redundant, essentially replicating the processes from the historic loading phase. This approach, while functional on the outside, did not fully adhere to the best and correct implementation of the incremental updates. Our audit phase further verified those issues. These findings will guide the necessary adjustments to improve both the correctness and efficiency of the TPC-DI benchmark moving forward.

4.6 Optimization

In this subsection, we will explain how we optimized the process improving like this the performance.

1. **Customer XML to CSV:** The ETL process defined by our predecessors used a non-optimal XML to CSV transformation. They were updating a pandas dataframe in a loop which is quite slow. We used an array for retrieving and preparing the data in the loop which performs better, and then transformed it into a pandas dataframe, getting better performance by avoiding multiple dataframe updates.

2. **Syntax optimization:** Some SQL files were not using optimal syntax making them slower. We optimized the syntax of those queries by rewriting them, using equivalent optimal syntax. For example, many queries were using long string concatenations with the operator —— and btrims which we replaced by CONCAT_WS, which is much faster due to better NULL handling.

```
--Old version
nullif(btrim(btrim(
case
    when p.networth > 1000000 or p.income > 200000
    then 'HighValue'
    else ''
end
|| '+' ||
case
    when p.numberchildren > 3 or p.numbercreditcards > 5
    then 'Expenses'
    else ''
end
, '+')
|| '+' ||
case
    when p.age < 25 and p.networth > 1000000
    then 'Inherited'
    else ''
end
, '+'), '')
--Optimized version
nullif(CONCAT_WS('+',
CASE WHEN p.networth > 1000000 OR p.income > 200000
THEN 'HighValue' END,
CASE WHEN p.numberchildren > 3 OR p.numbercreditcards > 5
THEN 'Expenses' END,
CASE WHEN p.age < 25 AND p.networth > 1000000
THEN 'Inherited' END
), '')
```

3. **Query optimization:** We tackled the main performance bottlenecks in the tasks as they were causing exponential behavior. A good example for the kind of optimization we did is the `12_transform_load_master_financial.sql` task, which was previously running for 1434 seconds on scale factor 10 with heavy exponential behavior. We split the conditions into two part using UNION ALL for (`companyid` OR `name`), which avoids the expensive OR logic in a single join. With such a simple optimization step, the task now runs for only 5 seconds on scale factor 10. Below is the example of this optimization:

Old Query

```
TRUNCATE TABLE master.financial;
INSERT INTO master.financial
SELECT
...
FROM staging.finwire_fin f,
     master.dimcompany c
WHERE (f.conameorcik =
       c.companyid::varchar
       OR f.conameorcik = c.name)
  AND LEFT(pts, 8)::date >=
       c.effectivedate
  AND LEFT(pts, 8)::date < c.enddate;
```

Optimized Query

```
TRUNCATE TABLE master.financial;
INSERT INTO master.financial (
...
)
SELECT
...
FROM processing.finwire_fin f
JOIN master.dimcompany c
    ON f.conameorcik = c.companyid::varchar
    AND LEFT(pts, 8)::date >= c.effectivedate
    AND LEFT(pts, 8)::date < c.enddate
UNION ALL
SELECT
...
FROM processing.finwire_fin f
JOIN master.dimcompany c
    ON f.conameorcik = c.name
    AND LEFT(pts, 8)::date >= c.effectivedate
    AND LEFT(pts, 8)::date < c.enddate;
```

Figure 12: Comparison of Old and Optimized Queries

4. **Indexing:** Another approach we tried out is putting indexes on columns used in multiple JOINs and window functions during inserts. On Figure 13 we have an example of this for `16_transform_load_master_factmarkethistory.sql` This improves the performance of this query by speeding up join operations, filtering, and window function computations. Temporary indexes created for specific columns (e.g., `dm_s_symb`, `dm_date`, `datevalue`, etc.) reduce the need for full table scans. We dropped them afterward to prevent interference with anything else and we had to use commits at every step to avoid issues. The performance was similar for scale factor 3, but improved for scale factor 10. However, we had to abandon this approach as even if we dropped them, they still caused conflicts when running the audit file.

```
BEGIN;
CREATE INDEX IF NOT EXISTS idx_dailymarket_symb_date
    ON processing.dailymarket (dm_s_symb, dm_date);

CREATE INDEX IF NOT EXISTS idx_dimdate_datevalue
    ON master.dimdate (datevalue);

CREATE INDEX IF NOT EXISTS idx_dimsecurity_symbol_dates
    ON master.dimsecurity (symbol, effectivedate, enddate);

CREATE INDEX IF NOT EXISTS idx_financial_qtrstart
    ON master.financial (sk_companyid, fi_qtr_start_date);
COMMIT;
BEGIN;
TRUNCATE TABLE master.factmarkethistory;
INSERT INTO master.factmarkethistory
WITH ...
INNER JOIN master.dimdate dd
    ON dm.dm_date = dd.datevalue
...
WHERE dm.dm_date >= s.effectivedate
  AND dm.dm_date < s.enddate
...
AND q.fi_qtr_start_date <= dm.dm_date
  AND q.next_qtr_start > dm.dm_date;
COMMIT;
BEGIN;
DROP INDEX IF EXISTS idx_dailymarket_symb_date;
DROP INDEX IF EXISTS idx_dimdate_datevalue;
DROP INDEX IF EXISTS idx_dimsecurity_symbol_dates;
DROP INDEX IF EXISTS idx_financial_qtrstart;
COMMIT;
```

Figure 13: Indexing example for `factmarkethistory`

4.7 Other small fixes

In the following paragraphs, we focus on the small changes we made to solve some bugs we found in the old queries.

4.7.1 LTRIM

When inserting into the **dimsecurity** table, you have to connect every row, to a company. You do so by using the field **conameorcik** of the finwire data.

As the name suggests this field can contain both the name or the company id, and because the company id starts with 0s, we can trim from the left all 0s to get the result, and then check if we have a company name or a company id, that matches this field.

The issue with using ltrim, can be seen in this figure 14

sk_companyid	companyid	status	name	industry
1	0	Active	kkhSvCsfgbDUvsiZWtPQwjkYPwKrfTUFMGMCWCBXBcOcTkaLSudO	Biotechnology & Drugs
2	1	Active	TGfFOOldnTHLjuAaQqJBgbibktVyiLStkrLCynHAZNCp	Metal Mining
3	2	Active	jRhDJJDxLPxUEMMpdhzdZNqRVTuzMCrlMcwMsdHNvIEBwDfErPG	Broadcasting & Cable TV
4	2	Inactive	jRhDJJDxLPxUEMMpdhzdZNqRVTuzMCrlMcwMsdHNvIEBwDfErPG	Broadcasting & Cable TV

Figure 14: Result of selection from master.dimcompany

There is a company whose id is 0, and we found, that while trimming a match was lost, because ltrim(conameorcik, '0'), returned an empty string.

We fixed that by adding a condition that checked if this exact edge case was happening.

4.7.2 Coalesce

We've noticed that the task **transform_load_master_dimcustomer** produced rows, where the gender was first set and then in a subsequent row of the same customer, it was set to undefined.

The issue was due to a wrong utilization of the coalesce function, because in the select statement, the gender was set directly to "undefined" so the coalesce behind was never run.

This example can be observed also in other columns like the DimAccount.Status, we tried to fix it by changing a little bit the logic.

4.8 Audit Phase

Even though one of the original repositories included an audit phase, there was no documentation and instructions. This presented an opportunity for us to improve, document, and utilize it effectively. The Audit Phase begins immediately after the completion of the last Incremental Update phase. During this phase, each field in the audit data is loaded into the corresponding column of the Audit table, ensuring that the data aligns with its respective column name. It is critical to note that no modifications to the contents of the Data Warehouse are permitted during this phase, except for loading data into the Audit table as outlined in the specified clauses. Once the data is loaded, the audit query is executed to perform a series of tests on the Data Warehouse, validating the integrity and accuracy of the result set. For the Benchmark Run to be deemed valid, all tests must return an "OK" status.

The purpose of the Audit Phase is to validate the results following the historic load and incremental updates. The audit phase was executed for scale factor 5. Before starting the audit, all text files ending with "_audit.csv" for SF5 were inserted into

the `master.audit` table, with the data placed in the appropriate columns. However, this process uncovered significant issues and errors in the original repository's ETL process. By leveraging the audit query, these problems were identified, enabling our team to address and improve the existing solution effectively. This phase thus serves as a critical checkpoint to ensure data accuracy and process reliability.

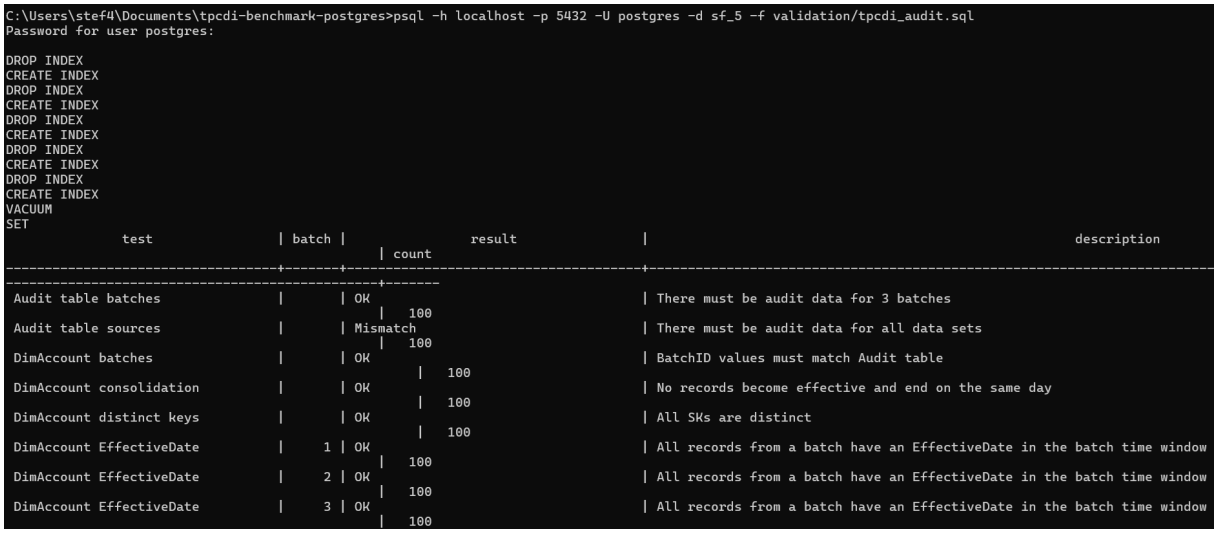


Figure 15: Audit Phase Run in Command Prompt

The table below displays all the audit checks that have been improved from the original repository to the final version. A total of 19 checks, which previously exhibited issues, have been resolved through debugging and updates to the ETL process, as shown. After a thorough examination, we successfully passed 100 out of 152 audit phase checks, significantly improving the accuracy and completeness of the TPC-DI implementation.

Test	**Description**	**Original Result**	**Final Result**
DimAccount batches	BatchID values must match Audit table	Mismatch	OK
DimAccount IsCurrent	IsCurrent is 1 if EndDate is the end of time, else IsCurrent is 0	Not current	OK
DimAccount row count	Actual row count matches or exceeds Audit table minimum	Too few rows	OK
DimCompany End of Time	Every company has one record with a date range reaching the end of time	End of time not reached	OK

DimCompany Overlap	Date ranges do not overlap for a given company	Dates overlap	OK
DimCustomer IsCurrent	IsCurrent is 1 if EndDate is the end of time, else IsCurrent is 0	Not current	OK
DimCustomer Overlap	Date ranges do not overlap for a given Customer	Dates overlap	OK
DimCustomer TaxID	TaxID values are properly formatted	Mismatch	OK
DimSecurity End of Time	Every company has one record with a date range reaching the end of time	End of time not reached	OK
DimSecurity IsCurrent	IsCurrent is 1 if EndDate is the end of time, else IsCurrent is 0	Not current	OK
DimSecurity Overlap	Date ranges do not overlap for a given company	Dates overlap	OK
DimTrade batches	BatchID values must match Audit table	Mismatch	OK
FactCashBalances batches	BatchID values must match Audit table	Mismatch	OK
FactHoldings batches	BatchID values must match Audit table	Mismatch	OK
FactMarketHistory batches	BatchID values must match Audit table	Mismatch	OK
FactWatches batches	BatchID values must match Audit table	Mismatch	OK
FactWatches SK_CustomerID	All SK_CustomerIDs match a DimCustomer record with a valid date range	Bad join	OK
FactWatches SK_SecurityID	All SK_SecurityIDs match a DimSecurity record with a valid date range	Bad join	OK
Prospect batches	BatchID values must match Audit table	Mismatch	OK

The following subsections will cover the most significant checks and optimizations of the audit phase:

4.8.1 FactWatches SK CustomerID & SK SecurityID

The FactWatches table is a fact table in a data warehouse schema that tracks transactional data related to customer activity on securities watch lists. These watch lists consist of securities that customers are monitoring or interested in for potential future transactions. The table records interactions or changes in the watch list, such as the addition or removal of securities.

A key detail is that securities can only be added to or removed from a watch list. There is no concept of updating a security on the watch list. The previous work overlooked this aspect. Additionally, it did not correctly match the FactWatches table with the DimCustomer and DimSecurity tables. After addressing and correcting these issues, the audit test status was updated to "OK" in the final results. Since there are no updates according to the TPC-DI manual, the same process was used in the incremental updates phase. In conclusion, we identified customer activities related to securities on a watch list (whether added or removed) and inserted records into the FactWatches table. This process involved multiple joins to retrieve surrogate keys from the dimension tables for customers, securities, and dates, ensuring that only valid records were inserted.

4.8.2 DimCustomer TaxID

In the `master.dimcustomer` table, we observed that all values in the `taxid` column were stored as a tax ID number followed by multiple trailing spaces, for example, `"923-54-6498 "`. To address this, we initially applied the `RTRIM()` function to remove the trailing spaces when the data was inserted in the table `master.dimcustomer`. However, this preprocessing had little effect because the original `taxid` column was defined as `taxid char(20) not null`, which automatically pads the remaining characters to fill 20 spaces.

We resolved the issue by redefining the `taxid` column as `varchar(20)`, eliminating the padding problem. This issue was identified in the audit phase and fixed to allow better filtering of tax-ids if needs be.

4.8.3 DimSecurity IsCurrent

The specific validation pertains to the `master.dimsecurity` table. In the previous process, the `IsCurrent` check for `dimsecurity` failed because the `enddate` column was calculated using a windows function lead on the batchdate of the table `staging.batchdate`. This resulted in a validation test failure, as the `enddate` should be computed using the lead windows function on the `pts` column of table `processing.finwire_sec`, which represents the posting date and time of the financial market information.

4.8.4 Batches Checks

All the checks involving the "batches" keyword—such as DimAccount batches, DimTrade batches, FactCashBalances batches, FactHoldings batches, FactMarketHistory batches, FactWatches batches, and Prospect batches—have been improved to account for the incremental update phase. These checks ensure that the two incremental updates are correctly implemented in each respective table.

4.9 Batch Validation Query

The batch validation query is designed to capture information for timing and verifying the correctness of each execution phase. It is executed multiple times during a Benchmark Run, and when used in a timed phase, the execution time is recorded. The query writes results into the DImessages table, which are later used in the automated audit phase to validate transformed data in the Data Warehouse. Additionally, according to the TPC-DI manual, it is compatible with most databases that support SQL queries.

Despite its potential to efficiently address errors detected during the audit phase, the batch validation query proved challenging to use due to a lack of sufficient documentation. Although we ran the batch validation query after each phase, including historic and incremental loads, the results did not align with any of the expected validation outputs. This misalignment created significant difficulty in interpreting the results and resolving identified issues, ultimately hindering the validation process and shifting our focus to the improvement of the correctness of the ETL process.

5 Results and discussion

In this section, we will explain the results obtained from the benchmark.

This benchmark evaluates two types of metrics simultaneously: performance and data validation. In this section, we will examine both for each scale factor. Performance measures the execution time for every task, while data validation ensures that the proposed ETL process returns the expected database results.

5.1 Performance results

In this subsection, we will showcase the performance results of our work. We created a Power BI dashboard that aggregates the results we measured, which is in our GitHub repository. The reader is invited to inspect the results further at this Power BI dashboard.

One of the most alarming things we noticed from previous works is the exponential behavior when increasing the scale factors. Thanks to our optimizations mentioned at Subsection 4.6, we were able to tackle this for the historical load. Figure 16 displays the total run time for our historical DAG by scale factor. The behavior is now close to linear.

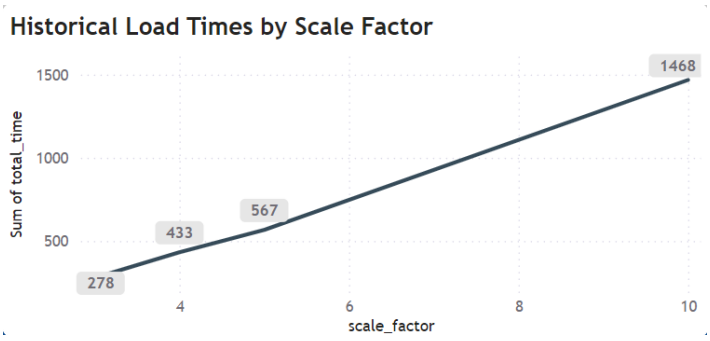
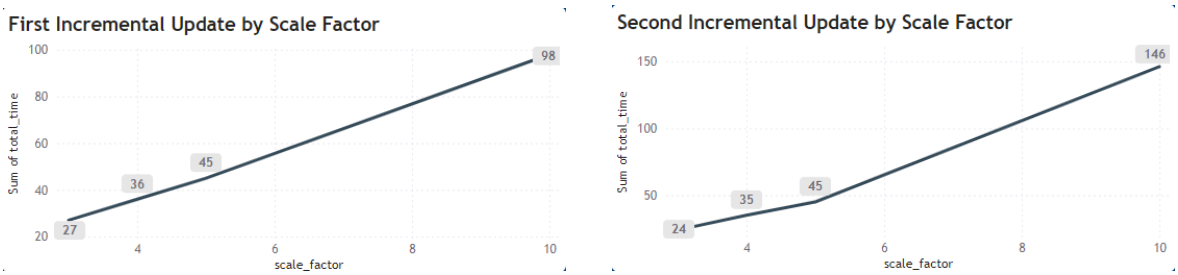


Figure 16: Total run time for our historical DAG by scale factor

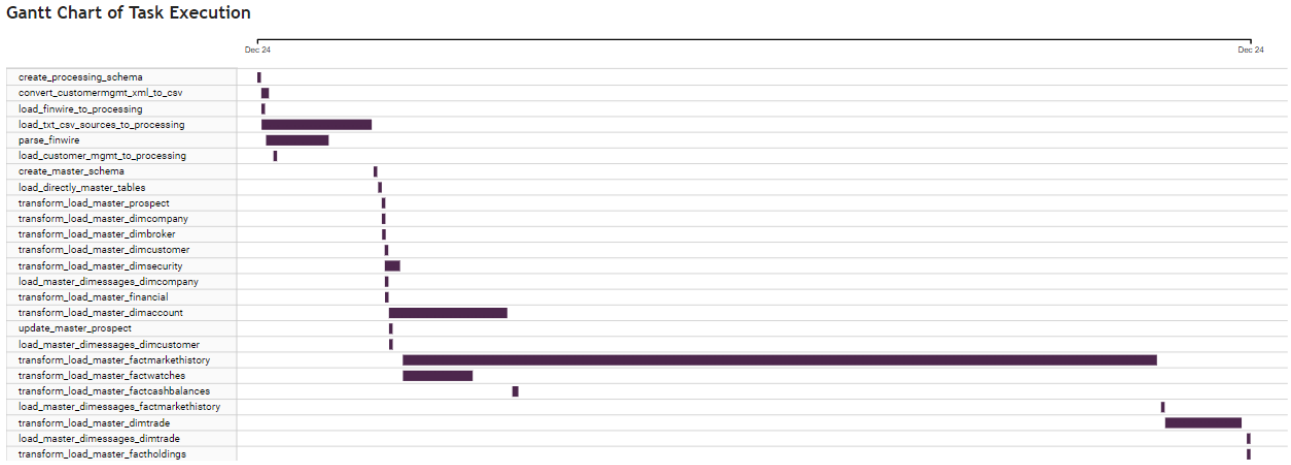
Both incremental refreshes show similar behavior, with the second incremental refresh being a bit closer to exponential. Future work can include tackling this behavior, even if load times are very fast in this case. The current load times are visible in Figure 17.

Lastly, we present the Gantt charts for both historical and incremental refreshes. We extracted the data for this from Airflow's database of the DAG runs using a custom Python script, which is also available in our repository. We only present these charts for scale factor 3, but they are available for all scale factors in our dashboard.

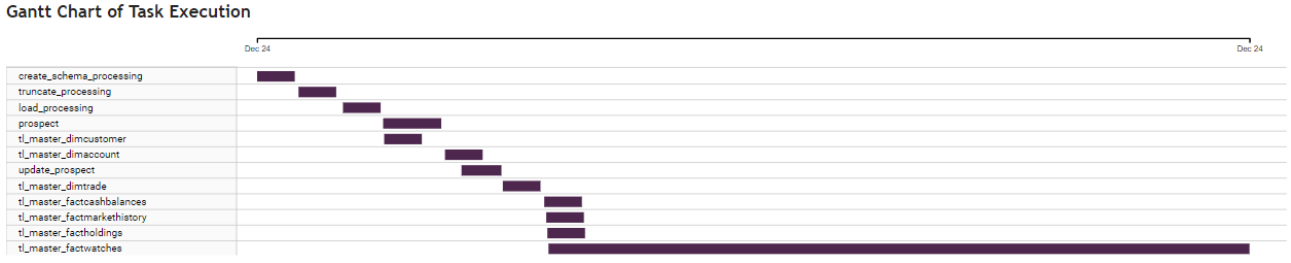


(a) First incremental update (b) Second incremental update

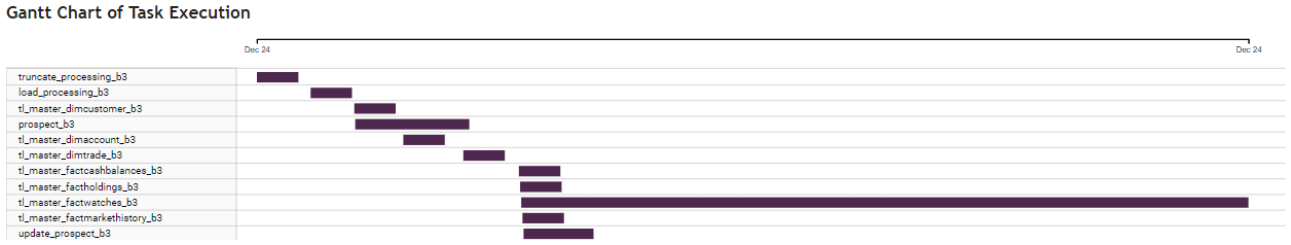
Figure 17: Incremental refresh times by scale factor



(a) Historical Gantt chart



(b) First incremental refresh Gantt chart



(c) Second incremental refresh Gantt chart

Figure 18: Gantt charts for historical and incremental refreshes for scale factor 3

Future work on historical load optimization should focus on both `transform_load_master_factmarkethistory` and the `transform_load_master_dimaccount` tasks. As mentioned before, we tried multiple approaches for the former, but they either did not lead to great performance improvements or conflicted with the audit. Figure 19 showcases the top 5 tasks by load time for the historical loads.

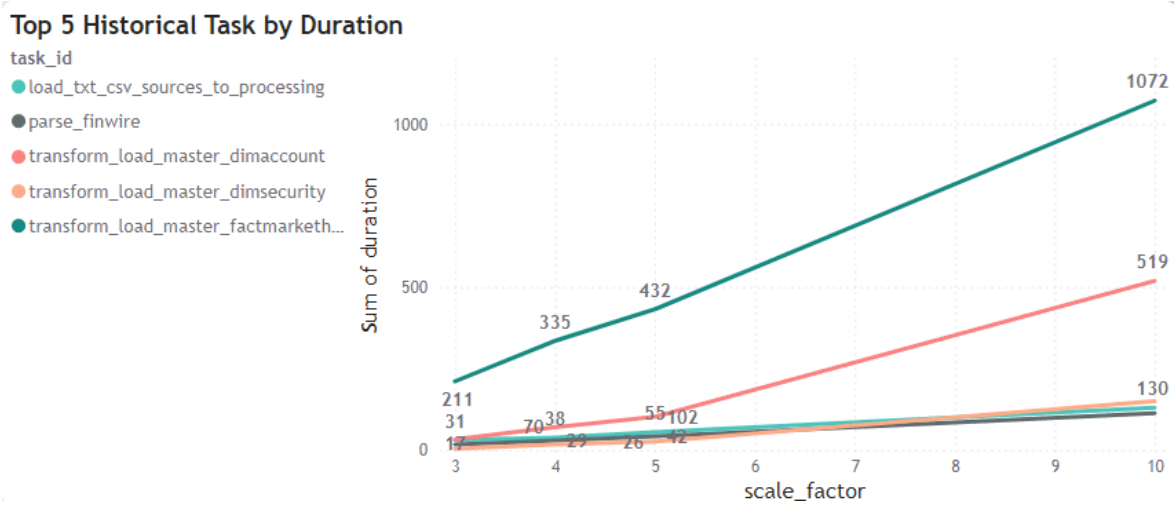


Figure 19: Top 5 tasks by load time for the historical loads.

6 Conclusion

In conclusion, this project successfully conducted a TPC-DI benchmark using PostgreSQL, providing valuable insights into the database's performance and scalability for Data Integration processes.

The results highlight PostgreSQL's effectiveness as a robust solution for data warehousing applications, particularly after implementing proper database optimization techniques. Furthermore, we have enhanced the optimization efforts from the previous year by improving execution time, efficiency, and ease of setup. Additionally, we have successfully combined historic and incremental updates, strengthened the audit phase, and ensured the correctness and completeness of the ETL process.

We have also reviewed the previous work and believe we have made significant improvements to the overall process and setup. However, we feel there is still more to be done, and we encourage future students to build upon our progress, focusing on areas we were unable to correct or enhance due to time constraints.

We suggest for example restructuring the historical load for those tables that require history tracking in case an alternative solution can't be found. Additionally, increasing the 100 successful audit checks that we have achieved to a number closer to 152, which represents the total number of audit checks. Another improvement would be the successful implementation of the validation batch queries, which would further boost the number of successful audit checks. Finally, the successful implementation of Data Visibility Queries to verify the visibility of data written to the Data Warehouse can be explored.

References

[1] Ahmad et al. *tpc-ds-benchmark*. Oct. 2022. URL: https://github.com/risg99/tpc-ds-benchmark.

[2] CBQu. *TPC-DI Benchmark*. Oct. 2022. URL: https://github.com/CBQu/TPC-DI.

[3] Laurenz Albe. "Bulk load performance in PostgreSQL". In: *CYBERTEC PostgreSQL | Services & Support* (Mar. 2024). URL: https://www.cybertec-postgresql.com/en/bulk-load-performance-in-postgresql.

[4] *TPC Current Specs*. [Online; accessed 24. Dec. 2024]. Dec. 2024. URL: https://www.tpc.org/tpc_documents_current_versions/current_specifications5.asp.

[5] Pavan Patibandla. *How a single PostgreSQL config change improved slow query performance by 50x*. URL: https://amplitude.engineering/how-a-single-postgresql-config-change-improved-slow-query-performance-by-50x-85593b8991b0.

[6] PostgreSQL. *PostgreSQL - Partitioning*. URL: https://www.postgresql.org/docs/current/ddl-partitioning.html.

[7] Hironobu Suzuki. *Cost Estimation in Single-Table Query*. URL: https://www.interdb.jp/pg/pgsql03/02.html.

[8] TPC. *TPC-DS*. URL: https://www.tpc.org/tpcds/.